

OpenText ALM Octane

Modern quality and delivery management with backlog, tests, defects, pipelines and Agile/DevOps traceability.

What it is

ALM Octane integrates quality and software delivery management around Agile/DevOps. It also appears in today's OpenText Core Software Delivery Platform ecosystem.

Official site and documentation

Use the official OpenText ALM Octane source as the primary reference:

<https://www.opentext.com/products/saas/core-software-delivery-platform>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

Workspace and backlog

Teams work with workspaces, features, user stories, defects and tasks while QA tracks criteria, coverage and quality.

Manual and automated tests

Octane organizes manual and automated tests, suites, runs and results, with CI integrations bringing automation and pipeline data into the same view.

Pipelines and quality

Pipelines connect builds and results to planned work so QA can inspect failures, trends and delivery risks.

Migration and integration

Legacy environments may require migration from ALM/QC, Jira/Xray or other sources. Plan entity, user, field, attachment and history mappings.

Good practices

Model the workspace before importing data and connect quality to releases, pipelines and product goals.

Installation and configuration - overview

Always use the official website and documentation. Check prerequisites and compatible versions, install for your operating system, configure environment variables or PATH only when required, validate the installation with the recommended command or screen, and run a first project/test before integrating it into the QA workflow.

How to start safely

Start from the official OpenText ALM Octane site and documentation. Confirm prerequisites, supported versions, operating system and required permissions. Learn in a controlled environment before using it in a critical workflow.

First QA-oriented use

Pick one small, verifiable task in OpenText ALM Octane, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where OpenText ALM Octane fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether OpenText ALM Octane still delivers value as the product evolves.

Common issues and troubleshooting

When OpenText ALM Octane fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.